

Hybrid RAG over Tweets with Postgres, pgvector, and LangGraph

1. Goal

Build a general-purpose question-answering system over tweet data stored in Postgres.

The system must answer questions that require both:

- relational lookup: dates, ids, links, retweet counts, favorite counts, hashtags, mentions, sorting, grouping, filtering
- semantic lookup: understanding the meaning of tweet text, not just matching keywords

The system should stay general. It should not create domain-specific tables such as `book_claims`, `entities`, or `relations`. The book-related question is only an example of a broader class of semantic database questions.

2. Core Design

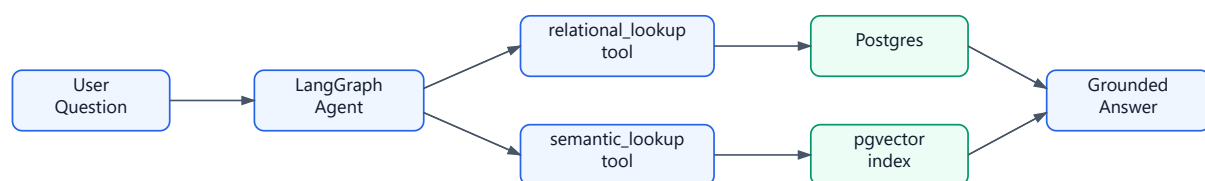
Use two Postgres tables:

```
tweets
tweet_embeddings
```

Use LangGraph to orchestrate an agent that can call two tools:

```
relational_lookup
semantic_lookup
```

The agent decides when to use exact SQL-style lookup, semantic vector lookup, or both.



3. Data Model

3.1 tweets

Stores one row per tweet.

```
CREATE TABLE tweets (
  id text PRIMARY KEY,
  link text NOT NULL,
  content text NOT NULL,
  created_at timestamptz NOT NULL,
  retweets integer,
  favorites integer,
  mentions text[],
  hashtags text[],
  geo text,
  raw_payload jsonb NOT NULL DEFAULT '{} '::jsonb
);
```

Notes:

- `id` is `text`, not numeric. Some tweet ids may be loaded from CSV in scientific notation or may exceed safe integer handling in application code. `**IDs should be normalized or fixed in the ingestion process**`
- `content` is the main semantic field.
- `created_at`, `retweets`, `favorites`, `mentions`, and `hashtags` support relational filtering.
- `raw_payload` preserves the original record for auditability and future fields.

Recommended indexes:

```
CREATE INDEX tweets_created_at_idx ON tweets (created_at);
CREATE INDEX tweets_retweets_idx ON tweets (retweets);
CREATE INDEX tweets_favorites_idx ON tweets (favorites);
CREATE INDEX tweets_mentions_gin_idx ON tweets USING gin (mentions);
CREATE INDEX tweets_hashtags_gin_idx ON tweets USING gin (hashtags);
CREATE INDEX tweets_content_trgm_idx ON tweets USING gin (content gin_trgm_ops);
```

`pg_trgm` is optional but useful for fallback lexical search and title/name matching.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

3.2 tweet_embeddings

Stores vector representations of tweet text.

```
CREATE TABLE tweet_embeddings (
  tweet_id text NOT NULL REFERENCES tweets(id) ON DELETE CASCADE,
  embedding_model text NOT NULL,
  embedded_text text NOT NULL,
  embedding vector(1536) NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tweet_id, embedding_model)
);
```

The vector dimension depends on the embedding model. Adjust `vector(1536)` to match the chosen model.

Recommended vector index:

```
CREATE INDEX tweet_embeddings_vector_idx
ON tweet_embeddings
USING hnsw (embedding vector_cosine_ops);
```

For smaller POCs, exact search without an index is acceptable. For production-scale data, use HNSW or IVFFlat.

4. Ingestion Pipeline

The ingestion pipeline loads CSV rows, normalizes them, stores tweets, embeds tweet text, and stores embeddings.



Algorithm:

```
for each CSV row:
    skip blank separator rows
    read tweet id as string
    if id is malformed or scientific notation:
        extract canonical id from tweet link
    parse content, link, date, retweets, favorites
    normalize mentions and hashtags into arrays
    insert/update tweets row

for each tweet without an embedding for the selected model:
    embedded_text = content
    embedding = embedding_model.embed(embedded_text)
    insert/update tweet_embeddings row
```

For a general system, embed the tweet `content` only. Do not inject task-specific labels into the embedded text. If later needed, a second embedding model/version can embed enriched text such as:

```
Tweet date: 2014-04-09
Content: ...
Mentions: ...
Hashtags: ...
```

That should be stored as another `embedding_model` or embedding profile.

5. Tool Contracts

The LangGraph agent should not directly access unrestricted database connections. It should call constrained tools.

5.1 relational_lookup

Purpose:

Use deterministic relational operations: filtering, counting, sorting, grouping, and fetching records by id.

Input shape:

```
{
  "operation": "select | count | aggregate | fetch_by_ids",
  "filters": {
    "created_at_from": "2009-01-01T00:00:00Z",
    "created_at_to": "2020-01-01T00:00:00Z",
    "mentions_contains": ["realDonaldTrump"],
    "hashtags_contains": ["trade"],
    "min_retweets": 100,
    "min_favorites": 100
  },
  "ids": ["1701461182"],
  "order_by": "created_at | retweets | favorites | engagement",
  "order_direction": "asc | desc",
  "limit": 50
}
```

Output shape:

```
{
  "rows": [
    {
      "id": "1701461182",
      "link": "https://twitter.com/realDonaldTrump/status/1701461182",
      "content": "Donald Trump will be appearing...",
      "created_at": "2009-05-05T03:00:10Z",
      "retweets": 33,
      "favorites": 273,
      "mentions": [],
      "hashtags": []
    }
  ],
  "row_count": 1
}
```

Implementation rule:

- Generate SQL inside the tool from structured parameters.
- Keep it read-only.
- Do not expose arbitrary SQL execution to the agent in the first POC.

5.2 semantic_lookup

Purpose:

Find tweets by meaning using `pgvector`, optionally constrained by relational filters.

Input shape:

```
{
  "query": "tweets where Trump says he wrote or authored a book",
  "filters": {
    "created_at_from": "2009-01-01T00:00:00Z",
    "created_at_to": "2020-01-01T00:00:00Z",
    "min_retweets": 0,
    "hashtags_contains": []
  },
  "top_k": 50
}
```

Output shape:

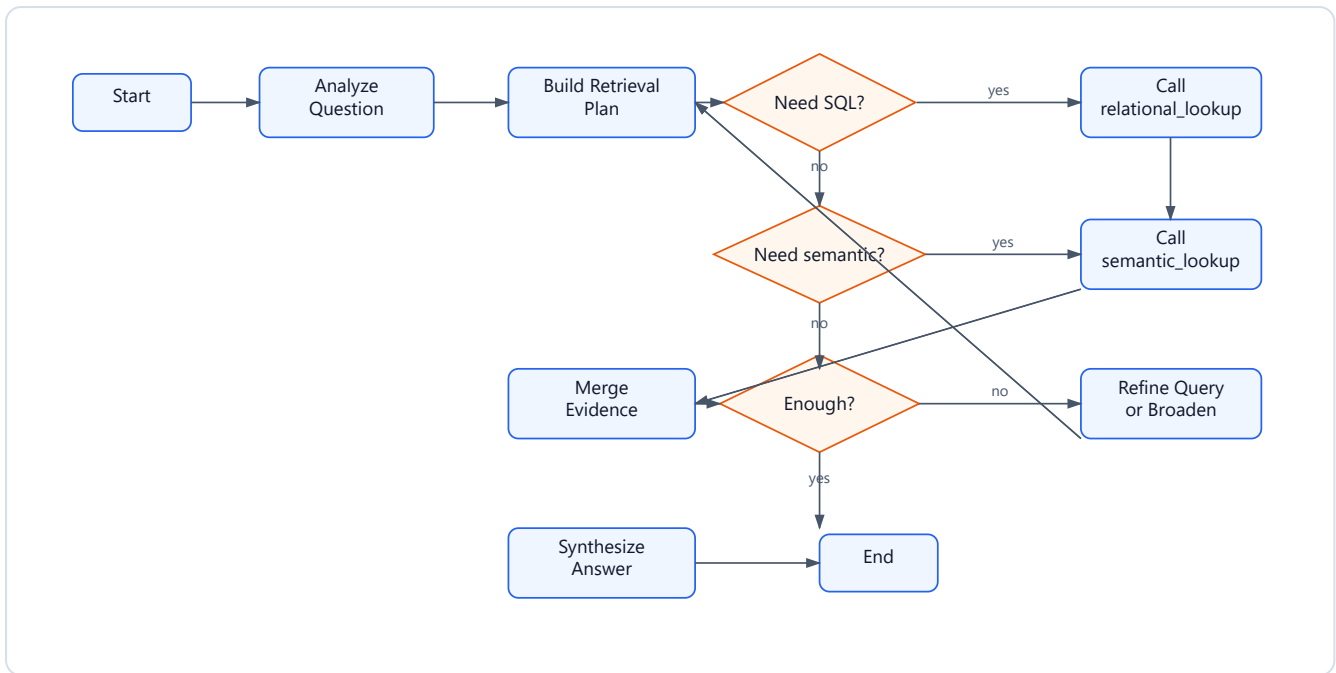
```
{
  "rows": [
    {
      "id": "1701461182",
      "link": "https://twitter.com/realDonaldTrump/status/1701461182",
      "content": "Donald Trump will be appearing...",
      "created_at": "2009-05-05T03:00:10Z",
      "similarity": 0.82
    }
  ]
}
```

Query pattern:

```
SELECT
  t.id,
  t.link,
  t.content,
  t.created_at,
  t.retweets,
  t.favorites,
  1 - (e.embedding <=> :query_embedding) AS similarity
FROM tweet_embeddings e
JOIN tweets t ON t.id = e.tweet_id
WHERE e.embedding_model = :embedding_model
  AND (:created_at_from IS NULL OR t.created_at >= :created_at_from)
  AND (:created_at_to IS NULL OR t.created_at < :created_at_to)
ORDER BY e.embedding <=> :query_embedding
LIMIT :top_k;
```

6. LangGraph Agent Design

Use LangGraph as a state machine rather than a free-form loop.



Suggested LangGraph state:

```

class AgentState(TypedDict):
    question: str
    retrieval_plan: dict
    relational_results: list[dict]
    semantic_results: list[dict]
    evidence: list[dict]
    attempts: int
    answer: str
  
```

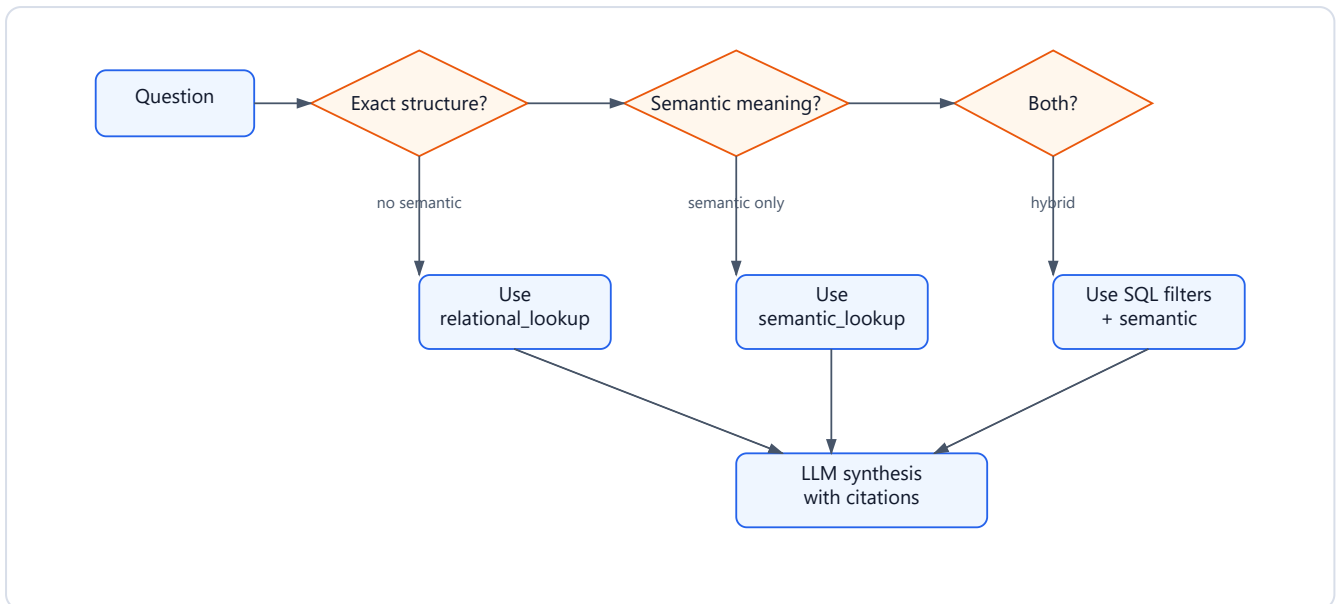
Nodes:

```

analyze_question
build_retrieval_plan
run_relational_lookup
run_semantic_lookup
merge_evidence
judge_evidence_sufficiency
refine_retrieval_plan
synthesize_answer
  
```

7. Query-Answering Algorithm

The agent should classify the question into one of four broad retrieval patterns.



7.1 SQL-only examples

Questions:

How many tweets are in 2019?
 What were the top 10 tweets by retweets?
 How many tweets mentioned @someone?

Algorithm:

```

parse structured constraints
call relational_lookup
return count/table/summary
  
```

7.2 Semantic-only examples

Questions:

What books does Trump talk about?
 What does Trump say about trade?
 Which tweets sound like campaign announcements?

Algorithm:

```

rewrite user question into retrieval query
call semantic_lookup with top_k = 50 or 100
cluster/deduplicate ideas in the retrieved evidence
answer with supporting tweet ids/links
  
```

7.3 Hybrid examples

Questions:

What did Trump say about books in 2014?
Which tweets about trade had the most engagement?
How many unique topics did he discuss after 2016 about China?

Algorithm:

```
extract structured constraints from the question
call semantic_lookup with those filters
optionally call relational_lookup on returned ids for sorting or aggregation
reason over retrieved evidence
return answer with citations
```

8. Evidence Handling

Every nontrivial answer should cite source tweets.

The agent should maintain evidence as:

```
{
  "tweet_id": "1701461182",
  "link": "https://twitter.com/realDonaldTrump/status/1701461182",
  "content": "Donald Trump will be appearing...",
  "created_at": "2009-05-05T03:00:10Z",
  "source": "semantic_lookup",
  "score": 0.82
}
```

For aggregation questions over semantic categories, the agent should explain uncertainty when needed:

```
I found two strongly supported titles in the retrieved tweets: ...
This is based on the top 50 semantic matches, not an exhaustive symbolic extraction over all
rows.
```

For a stronger POC, increase recall by running multiple semantic queries and merging results.

Example:

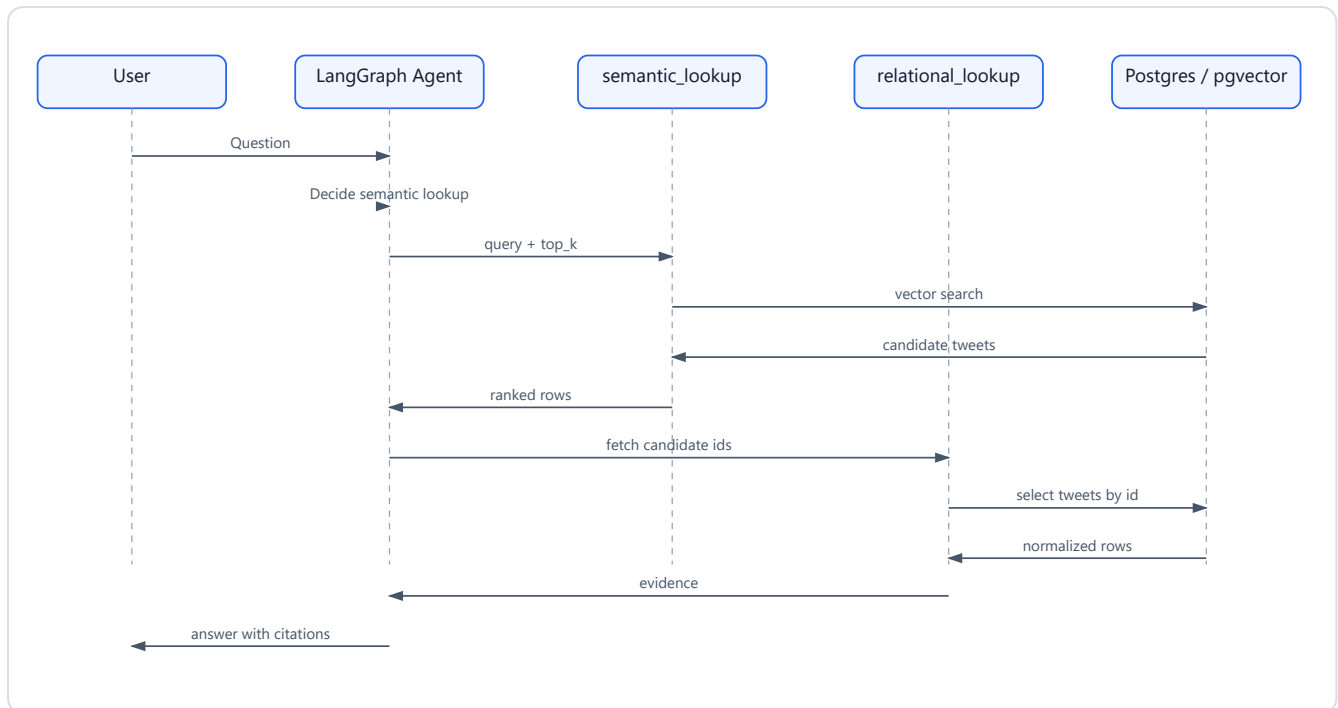
```
"Trump wrote a book"
"Trump authored a book"
"Trump's book titled"
"book by Donald Trump"
"read my book"
```

9. Example: Book Question

Question:

```
Tell me the names of the books Trump has written.
```

Execution:



Expected behavior on `selected_tweets.csv` :

Think Like A Champion
The Art of the Deal

Rows about generic book lists or Ivanka Trump's book should be rejected because they do not semantically support "Trump has written this book".

10. POC Implementation Plan

Phase 1: Database Setup

```

install Postgres extensions: vector, pg_trgm
create tweets table
create tweet_embeddings table
load trump_tweets.csv
load selected_tweets.csv for test cases
  
```

Phase 2: Embedding Pipeline

```

select embedding model
batch tweets without embeddings
store vectors in tweet_embeddings
verify vector search manually
  
```

Phase 3: Tools

```

implement relational_lookup with structured parameters
implement semantic_lookup with query embedding + optional filters
return compact JSON rows with ids, links, content, dates, and scores
  
```

Phase 4: LangGraph Agent

```
build graph nodes
connect lookup tools
add evidence sufficiency loop
add final answer synthesis with citations
```

Phase 5: Evaluation

Use `selected_tweets.csv` as a small semantic correctness test.

Test cases:

```
Tell me the names of the books Trump has written.
What tweets mention Ivanka's book?
Which tweets talk about Think Like A Champion?
Which book-related tweets are not about books Trump wrote?
```

Success criteria:

```
returns grounded answers with tweet links
uses relational filters correctly
uses semantic lookup for meaning-based questions
rejects obvious false positives
does not require task-specific database tables
```

11. Key Design Decisions

1. Keep the database model general.

Only `tweets` and `tweet_embeddings` are required. Do not create per-question extraction tables for the initial system.

2. Keep tools constrained.

The agent should call typed lookup tools, not arbitrary write-capable SQL.

3. Use hybrid retrieval.

SQL handles exact constraints. pgvector handles meaning. The LLM handles interpretation and synthesis over retrieved evidence.

4. Preserve evidence.

Answers should cite tweet ids or links so users can inspect the basis of the result.

5. Favor recall before synthesis.

For semantic aggregation questions, retrieve a larger candidate set, then deduplicate and classify in the agent's reasoning step.

12. Final Architecture Summary

CSV tweets

- > ingestion
- > Postgres tweets table
- > embedding generation
- > pgvector tweet_embeddings table
- > LangGraph agent
 - > relational_lookup for exact database operations
 - > semantic_lookup for meaning-based retrieval
 - > evidence merge and reasoning
- > cited natural language answer

This design remains general-purpose while still solving questions that traditional SQL or Text-to-SQL alone cannot answer.